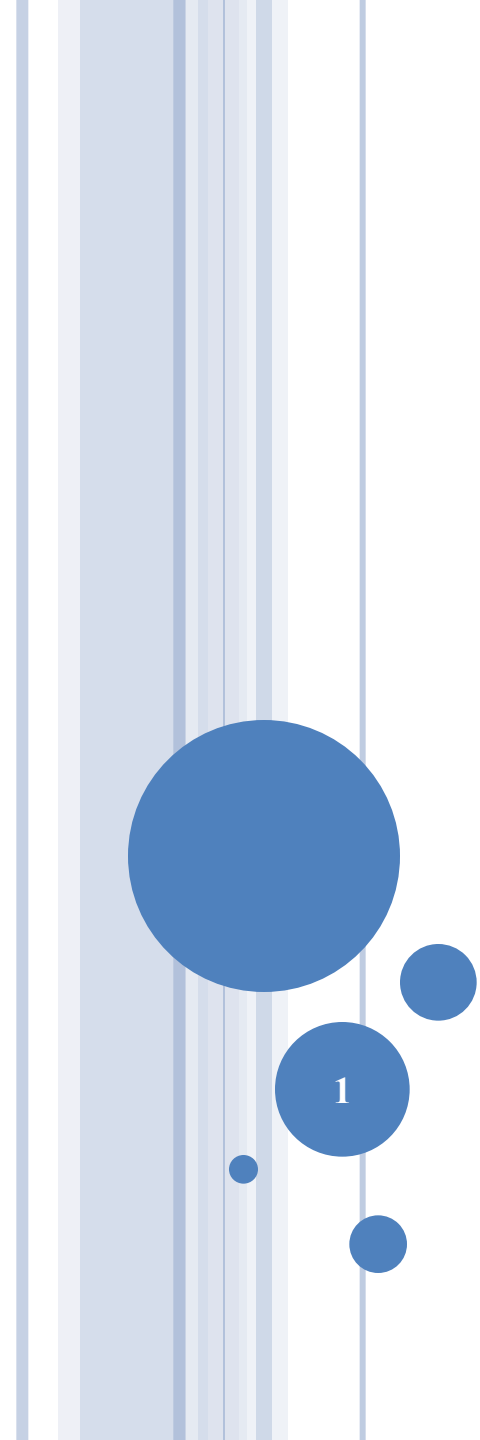




TRANSACTIONS

1

WHY TRANSACTIONS?

- Database systems are normally being accessed by many users or processes at the same time.
- Unlike OS, which *support* interaction of processes, a DMBS needs to keep processes from troublesome interactions.

EXAMPLE: BAD INTERACTION

- You and your partner each take \$100 from different ATM's at about the same time.
 - The DBMS better make sure one account deduction doesn't get lost.
- **Compare:** An OS allows two people to edit a document at the same time. If both write, then changes made by one of them are lost.

TRANSACTIONS

- *Transaction* = process involving database queries and/or modification.
- Normally with some strong properties regarding concurrency.
- Formed in SQL from single statements or explicit programmer control.

ACID TRANSACTIONS

□ *ACID transactions* are:

- *Atomic* : Whole transaction or none is done.
- *Consistent* : Database constraints preserved.
- *Isolated* : It appears to the user as if only one process executes at a time.
- *Durable* : Effects of a process survive a crash.

□ **Optional**: weaker forms of transactions are often supported as well.

COMMIT

- The SQL statement COMMIT causes a transaction to complete.
 - It's database modifications are now permanent in the database.

ROLLBACK

- The SQL statement ROLLBACK also causes the transaction to end, but by *aborting*.
 - No effects on the database.
- Failures like division by 0 or a constraint violation can also cause rollback, even if the programmer does not request it.

EXAMPLE: INTERACTING PROCESSES

- Consider a relation **Sells(shop, item, price)**
- Let Ahmad sells Sprite for Rs12.50 and Pizza Slize for Rs 35.00.
- Tania is querying Sells for the highest and lowest price Ahmad charges.
- Ahmad decides to stop selling Sprite and Pizza, but to sell Biryani at Rs75.00 per plate.

CUSTOMER'S PROGRAM

- Tania(customer) executes the following two SQL statements called **(min)** and **(max)** to help us remember what they do.

(max) SELECT MAX(price) FROM Sells
WHERE shop = 'Ahmad''s shop';

(min) SELECT MIN(price) FROM Sells
WHERE shop = 'Ahmad''s shop';

SHOP KEEPER'S PROGRAM

- At about the same time, Ahmad executes the following steps:
(del) and (ins).

(del) DELETE FROM Sells

WHERE shop = 'Ahmad''s shop';

(ins) INSERT INTO Sells

VALUES('Ahmad''s shop', 'Biryani', 75.00);

INTERLEAVING OF STATEMENTS

- The statement (max) must come before (min), and
- The statement (del) must come before (ins),
- There are no other constraints on the order of these statements, unless we group Tania's and/or Ahmad's statements into transactions.

EXAMPLE: STRANGE INTERLEAVING

- Suppose the steps execute in the order
(max)(del)(ins)(min).

Ahmad's Prices: {12.50,35.00} {12.50,35.00}{75.00}{75.00}

Statement:	(max)	(del)	(ins)	(min)
Result:	35.00			75.0

- Tania sees $MAX < MIN$!

FIXING THE PROBLEM BY USING TRANSACTIONS

- If we group Tania's statements (max)(min) into one transaction, then she cannot see this inconsistency.
- She sees Ahmad's prices at some fixed time.
 - Either before or after he changes prices, or in the middle, but the MAX and MIN are computed from the same prices.

ANOTHER PROBLEM: ROLLBACK

- Suppose Ahmad executes **(del)(ins)**, not as a transaction.
- But after executing these statements, he change his mind and issues a ROLLBACK statement.
- If Tania executes her statements after **(ins)** but before the rollback, she sees a value, 75.00, that never existed in the database.

SOLUTION

- If Ahmad executes **(del)(ins)** as a transaction, its effect cannot be seen by others until the transaction executes COMMIT.
 - If the transaction executes ROLLBACK instead, then its effects can *never* be seen.

TRANSACTION PROCESSING

- Basic operations in a DB are **read** and **write**
 - **read_item(X):**
 - Reads a database item named X into a program variable.
 - To simplify our notation, we assume that the program variable is also named X.
 - **write_item(X):**
 - Writes the value of program variable X into the database item named X.

SAMPLE TRANSACTIONS

(a) T_1

read_item (X);
 $X := X - N$;
write_item (X);
read_item (Y);
 $Y := Y + N$;
write_item (Y);

(b) T_2

read_item (X);
 $X := X + M$;
write_item (X);

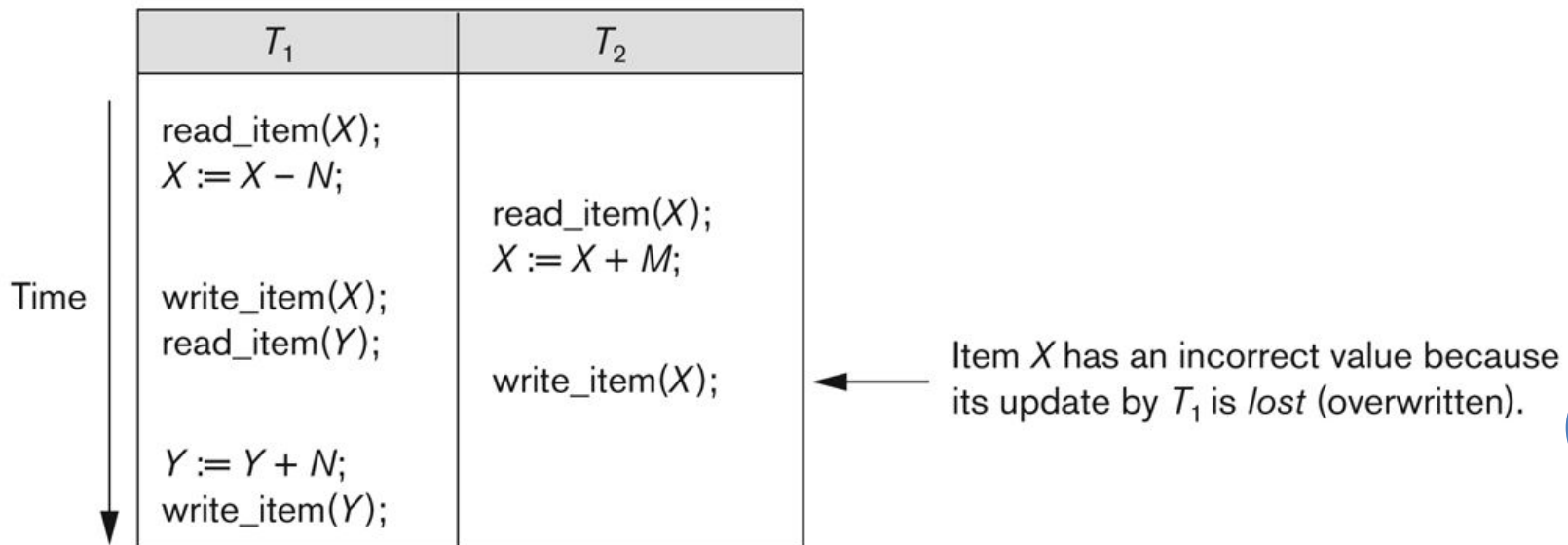


ISSUES IN TRANSACTION PROCESSING

Why Concurrency Control is needed:

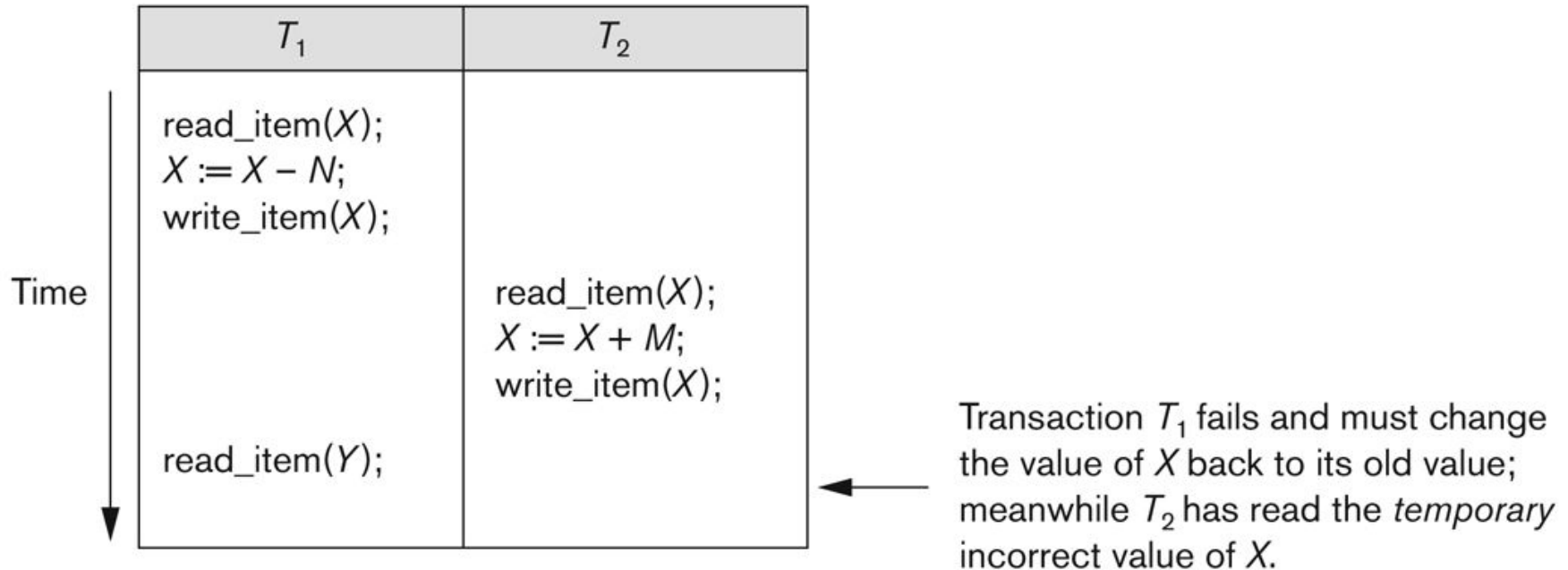
□ The Lost Update Problem

- Two transactions (that access the same DB items) have their operations interleaved in a way that makes the value of some database item incorrect.



ISSUES IN TRANSACTION PROCESSING

Temporary Update (or Dirty Read) Problem



❑ The Incorrect Summary Problem

← T_3 reads X after N is subtracted and reads Y before N is added; a wrong summary is the result (off by N).

TRANSACTION PROCESSING & RECOVERY

- Why **recovery** is needed:
 - A computer failure (system crash):
 - A transaction or system error:
 - Integer overflow, division by zero, erroneous parameter values or the user may interrupt the transaction
 - Local errors or exception conditions
 - Data not found or
 - insufficient account balance may cause a fund withdrawal transaction to be canceled.
 - Concurrency control enforcement
 - Transaction violates serializability or several transactions are in a state of deadlock
 - Disk failure
 - Physical problems and catastrophes

TRANSACTION AND SYSTEM CONCEPTS

- A **transaction** is an atomic unit of work that is either completed in its entirety or not done at all.
- For recovery purposes, the system needs to keep track of when the transaction starts, terminates, and commits or aborts
- **Transaction states:**
 - Active state
 - Partially committed state
 - Committed state
 - Failed state
 - Terminated State

TRANSACTION AND SYSTEM CONCEPTS

- Recovery manager keeps track of the following operations:
 - **begin_transaction**
 - **read or write**
 - **end_transaction**
 - **Commit**
 - **Rollback (or abort)**
 - **Undo**
 - Similar to rollback except that it applies to a single operation rather than to a whole transaction.
 - **Redo**

STATE TRANSITION DIAGRAM

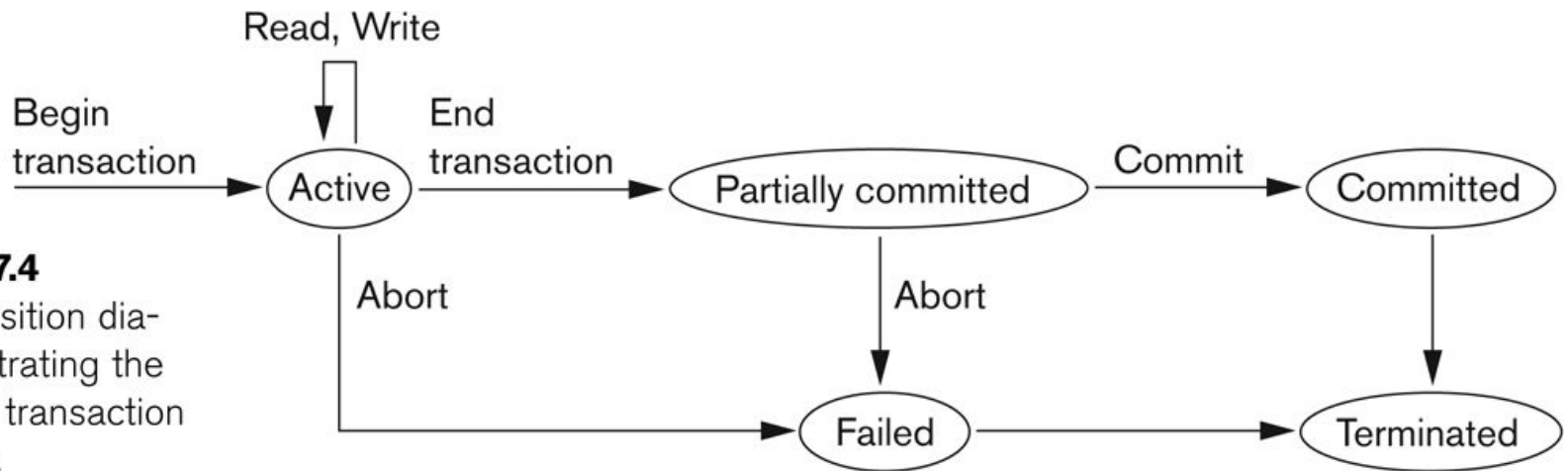


Figure 17.4

State transition diagram illustrating the states for transaction execution.

SCHEDULES

- A **schedule** (or **history**) S of n transactions $T_1, T_2, \dots, T_n$ is an ordering of the operations of the transactions.
- Operations from different transactions can be interleaved in S .
- The operations of each T_i in S must appear in the same order in which they occur in T_i .

	T_1	T_2
Time ↓	read_item(X); $X := X - N$;	read_item(X); $X := X + M$; write_item(X);
	write_item(X); read_item(Y);	
	$Y := Y + N$; write_item(Y);	

$S_a: r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); w_1(Y);$



SCHEDULES

- Two operations in S are in **conflict** if
 - they belong to *different transactions*;
 - they access the *same item* X ; and
 - *at least one* of the operations is a $\text{write_item}(X)$.

$S_2: r_1(X); w_1(X); r_2(X); w_2(X); r_1(Y); a_1;$

- Conflicting operations
 - $r1(X)$ and $w2(X)$
 - $r2(X)$ and $w1(X)$
 - $w1(X)$ and $w2(X)$
- Non-conflicting operations
 - $r1(X)$ and $r2(X)$
 - $w2(X)$ and $w1(Y)$
 - $r1(X)$ and $w1(X)$

Intuitively, two operations are conflicting if changing their order can result in a different outcome.

Types of Conflict: **read-write conflict** and **write-write conflict**

CHARACTERIZING SCHEDULES BASED ON RECOVERABILITY

- A schedule S is **recoverable** if no T in S commits until all T' that have written an item that T reads have committed.
- A transaction T **reads** from transaction T' in S if some item X is first written by T' and later read by T .
- In a recoverable schedule, no committed transaction ever needs to be rolled back

$S_a: r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); c_2; w_1(Y); c_1;$

$S_c: r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); c_2; a_1;$

$S_d: r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); w_1(Y); c_1; c_2;$

$S_e: r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); w_1(Y); a_1; a_2;$

Sc non recoverable

Sd sa and se are recoverable



CHARACTERIZING SCHEDULES BASED ON RECOVERABILITY (3)

▣ **Cascade rollback:**

- A schedule in which uncommitted transactions that read an item from a failed transaction must be rolled back.

$S_e: r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); w_1(Y); a_1; a_2;$

T_2 has to be rolled back because it read item X from T_1 , and T_1 then aborted.



CHARACTERIZING SCHEDULES BASED ON RECOVERABILITY

□ Cascadeless schedule

- One where every transaction reads only the items that are written by committed transactions.
- Avoid cascade rollback

$S_d: r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); w_1(Y); c_1; c_2;$

$S_e: r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); w_1(Y); a_1; a_2;$

- The $r_2(X)$ command in schedules S_d and S_e must be postponed until after T_1 has committed (or aborted),
- This delays T_2 but ensuring no cascading rollback if T_1 aborts



CHARACTERIZING SCHEDULES BASED ON RECOVERABILITY

□ Strict Schedules:

- A schedule in which a transaction can neither read or write an item X until the last transaction that wrote X has committed.
- Strict schedules simplify the recovery process.
- The process of undoing a **write_item(X)** of an aborted transaction is to restore the **before image** (old_value) of X .

● S_f is ca $S_f: w_1(X, 5); w_2(X, 8); a_1;$

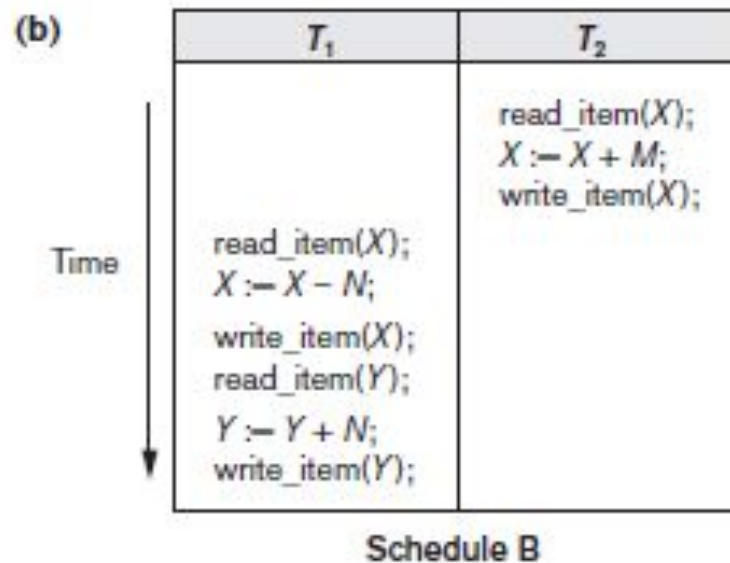
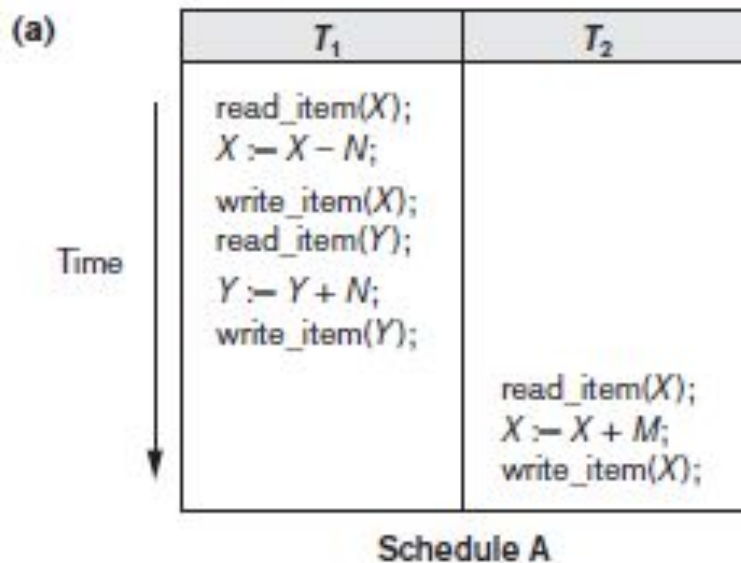
- As it permits T_2 to write item X even though T_1 that last wrote X had not yet committed (or aborted).



- It is important to note that any strict schedule is also cascadeless, and
- any cascadeless schedule is also recoverable

CHARACTERIZING SCHEDULES BASED ON SERIALIZABILITY

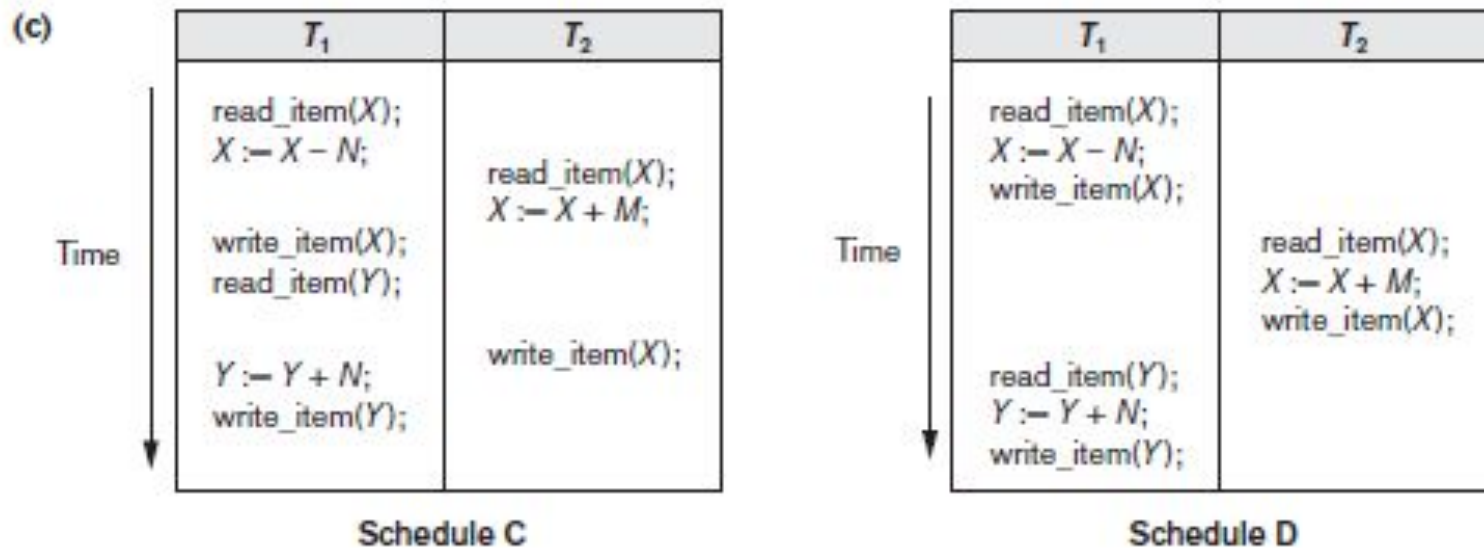
- **Serial schedule:** A schedule S is serial if, for every T in S, all the operations of T are executed consecutively in the schedule.



CHARACTERIZING SCHEDULES BASED ON SERIALIZABILITY

□ Non Serial schedule

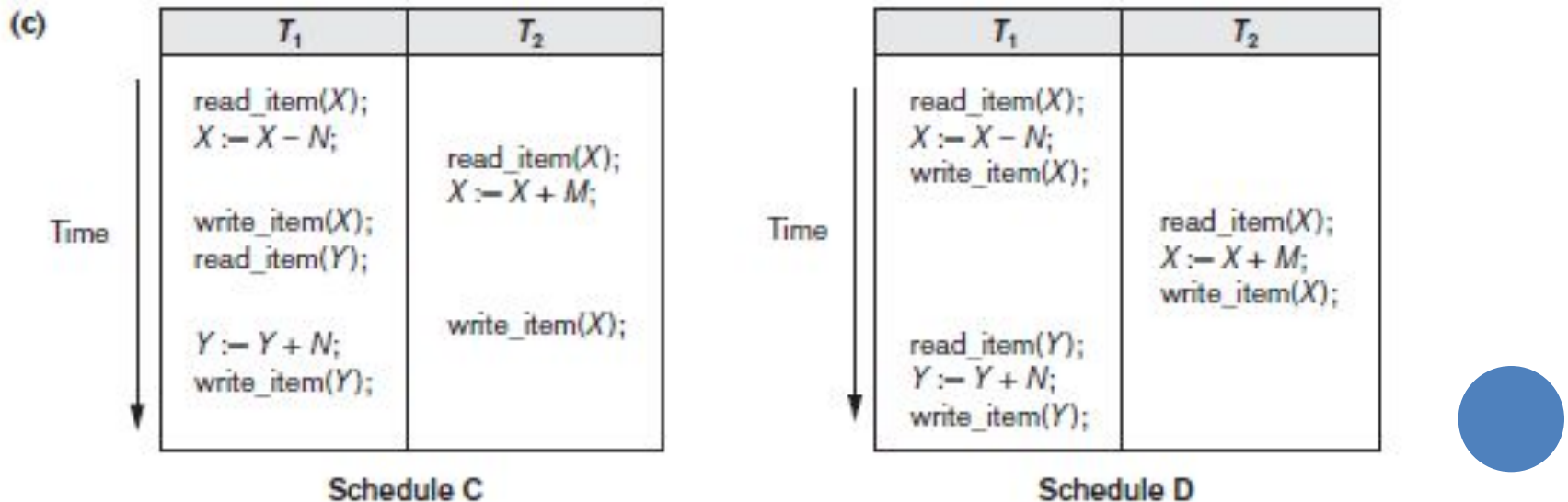
● Which one is correct ?



- Schedule C gives an erroneous result because of the *lost update problem*
- Schedule D gives correct results

CHARACTERIZING SCHEDULES BASED ON SERIALIZABILITY

- We would like to determine which of the non-serial schedules *always* give a correct result and which do not .
- **Serializable schedule:** A schedule S is serializable if it is equivalent to some serial schedule of the same n transactions.
 - A nonserial schedule S is serializable means it is correct,



CHARACTERIZING SCHEDULES BASED ON SERIALIZABILITY

- **Conflict equivalent:** Two schedules are said to be conflict equivalent if the order of any two conflicting operations is same in both schedules.

Two operations on the same data item conflict if at least one of the operations is a write

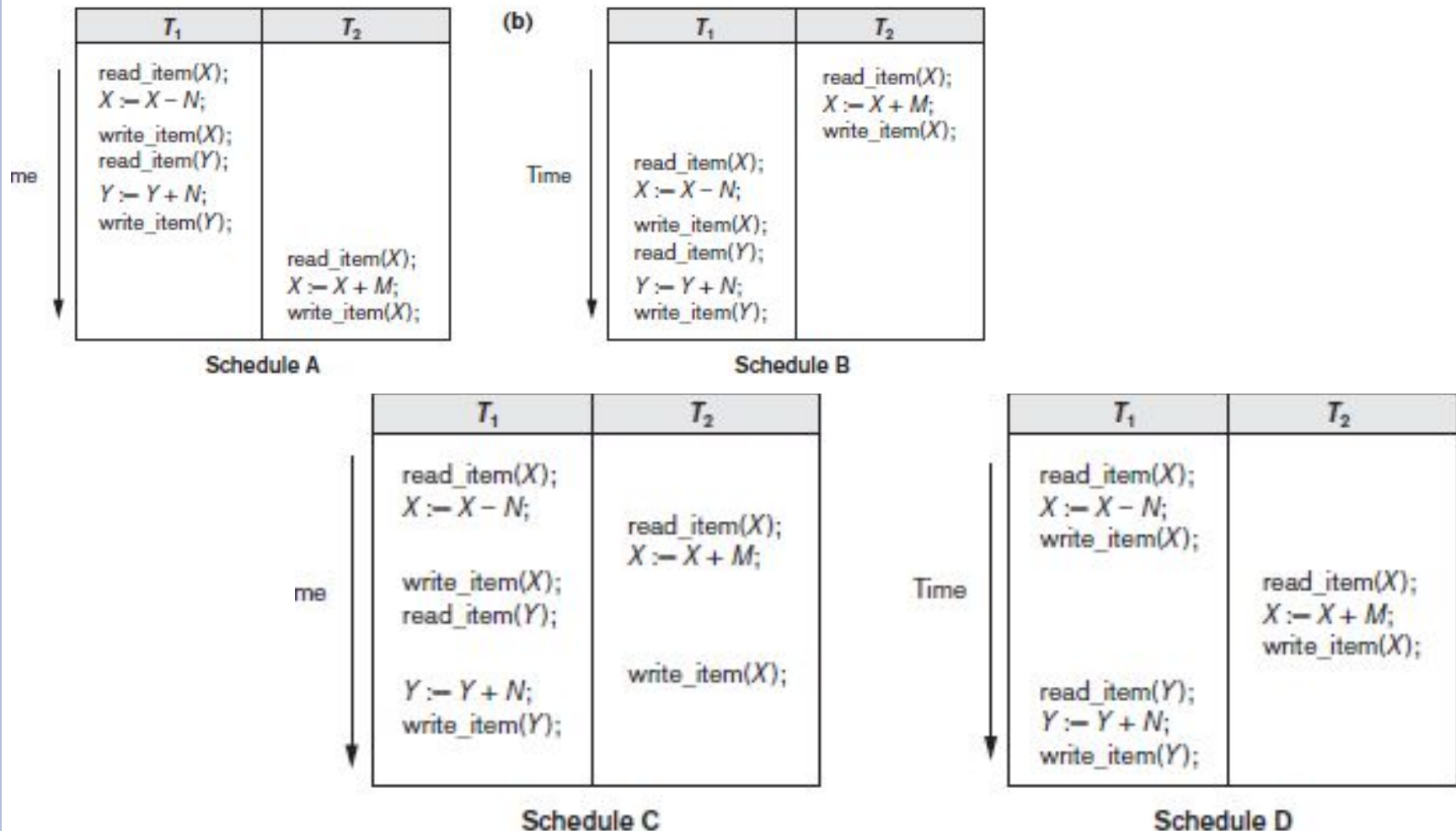
- $r(X)$ and $w(X)$ conflict
- $w(X)$ and $r(X)$ conflict
- $w(X)$ and $w(X)$ conflict

- If two conflicting operations are applied in *different orders* in two schedules, the effect can be different



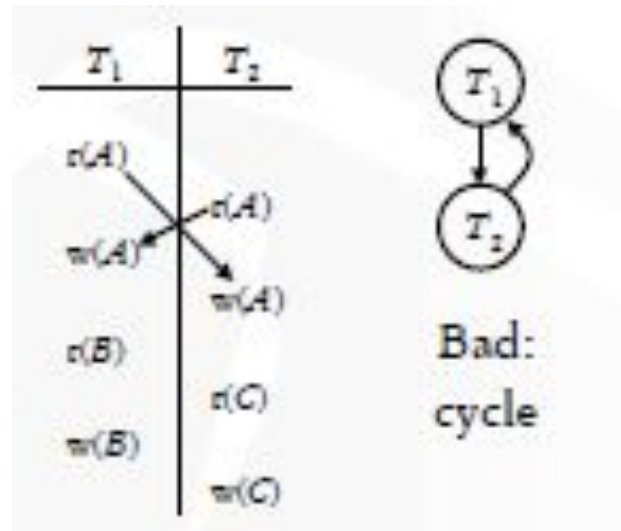
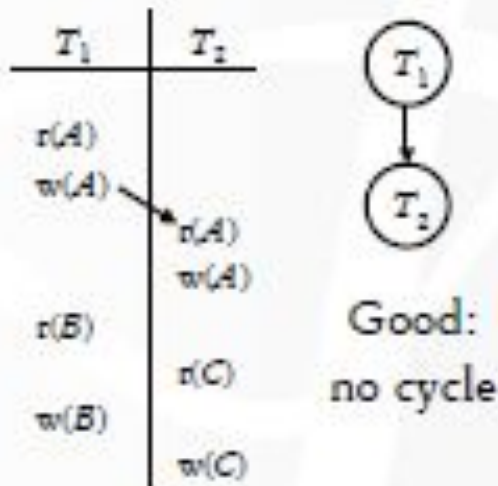
SERIALIZABILITY

- A schedule S is said to be conflict serializable if it is conflict equivalent to some serial schedule S'.



PRECEDENCE GRAPH

- A node for each transaction
- A directed edge from T_i to T_j if an operation of T_i precedes and conflicts with an operation of T_j in the schedule



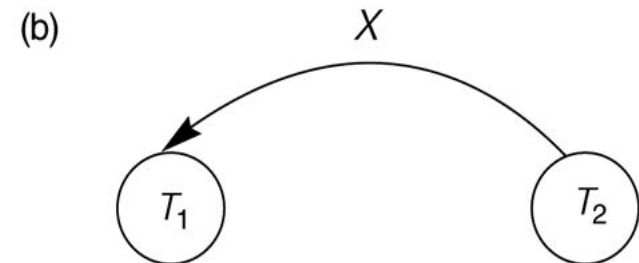
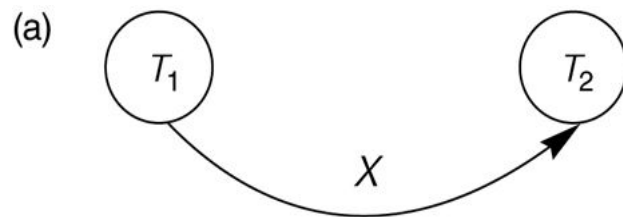
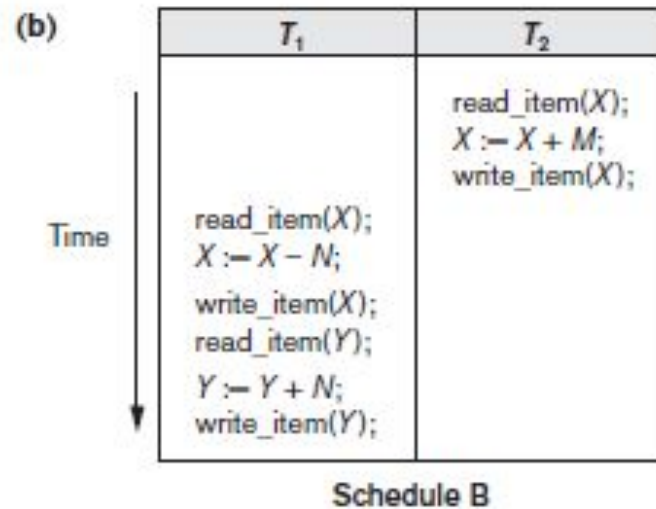
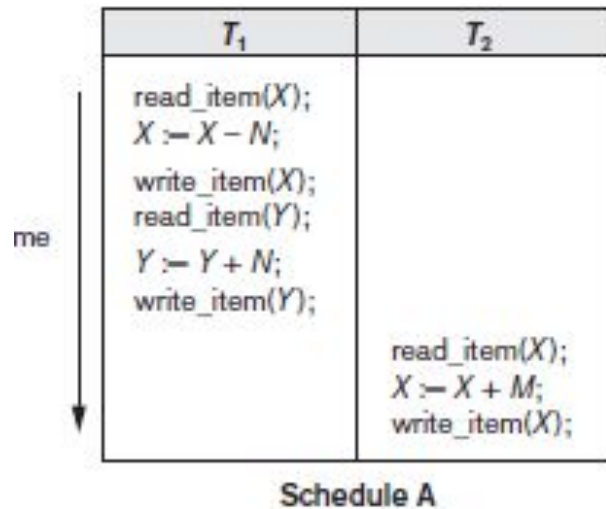
PRECEDENCE GRAPH

Testing for conflict serializability: Algorithm

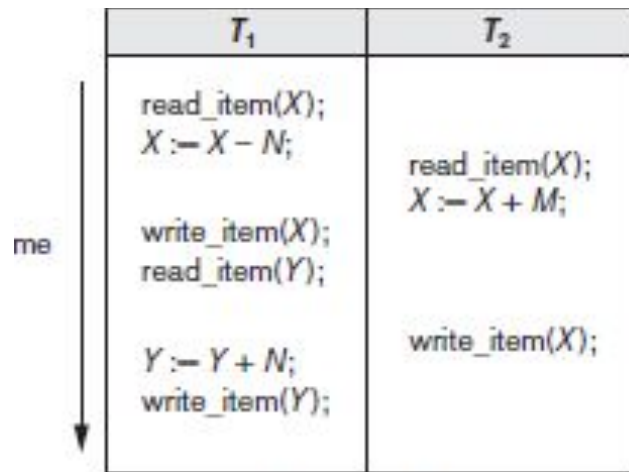
1. For each T_i in S , create a node labeled T_i in the precedence graph.
2. If T_j executes a `read_item( $X$ )` after T_i executes a `write_item( $X$ )`, create an edge ($T_i \rightarrow T_j$) in the precedence graph.
3. If T_j executes a `write_item( $X$ )` after T_i executes a `read_item( $X$ )`, create an edge ($T_i \rightarrow T_j$) in the precedence graph.
4. If T_j executes a `write_item( $X$ )` after T_i executes a `write_item( $X$ )`, create an edge ($T_i \rightarrow T_j$) in the precedence graph.
5. The schedule S is serializable if and only if the precedence graph has no cycles..



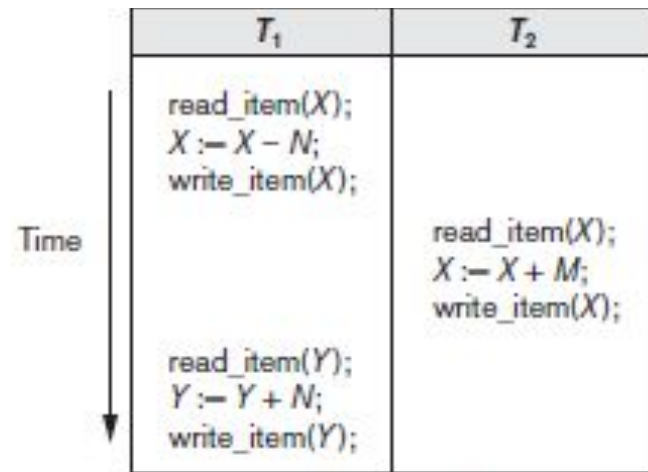
CONSTRUCTING THE PRECEDENCE GRAPHS



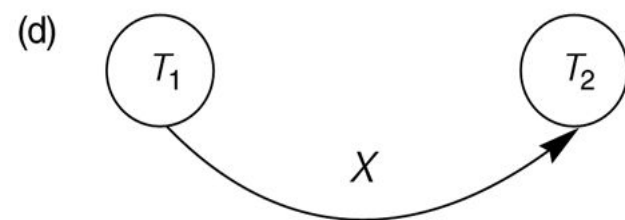
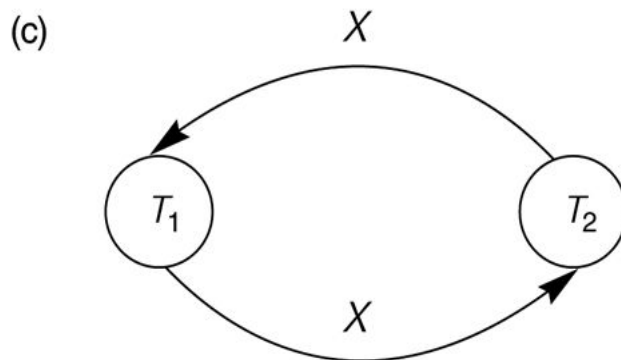
CONSTRUCTING THE PRECEDENCE GRAPHS



Schedule C



Schedule D



EXAMPLE OF SERIALIZABILITY TESTING

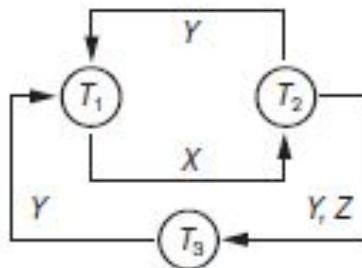
Transaction T_1	Transaction T_2	Transaction T_3
read_item(X); write_item(X); read_item(Y); write_item(Y);	read_item(Z); read_item(Y); write_item(Y); read_item(X); write_item(X);	read_item(Y); read_item(Z); write_item(Y); write_item(Z);



EXAMPLE OF SERIALIZABILITY TESTING

	Transaction T_1	Transaction T_2	Transaction T_3
Time ↓	read_item(X); write_item(X);	read_item(Z); read_item(Y); write_item(Y);	read_item(Y); read_item(Z);
	read_item(Y); write_item(Y);	read_item(X); write_item(X);	write_item(Y); write_item(Z);

Schedule E



Equivalent serial schedules

None

Reason

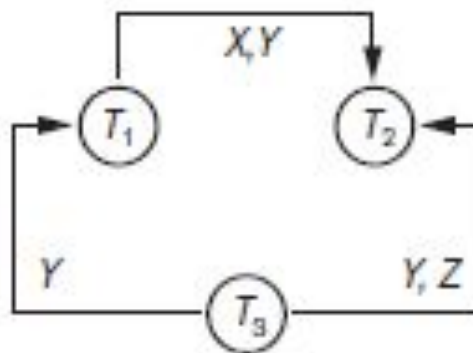
Cycle $X(T_1 \rightarrow T_2), Y(T_2 \rightarrow T_1)$

Cycle $X(T_1 \rightarrow T_2), YZ(T_2 \rightarrow T_3), Y(T_3 \rightarrow T_1)$

EXAMPLE OF SERIALIZABILITY TESTING

	Transaction T_1	Transaction T_2	Transaction T_3
Time ↓	read_item(X); write_item(X); read_item(Y); write_item(Y);	 read_item(Z); read_item(Y); write_item(Y); read_item(X); write_item(X);	read_item(Y); read_item(Z); write_item(Y); write_item(Z);

Schedule F

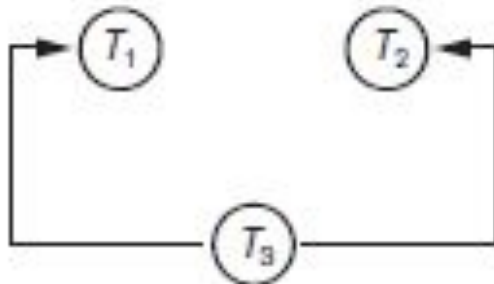


Equivalent serial schedules

$T_3 \rightarrow T_1 \rightarrow T_2$

EXAMPLE OF SERIALIZABILITY TESTING

Transaction T_1	Transaction T_2	Transaction T_3
read_item(X); write_item(X); read_item(Y); write_item(Y);	read_item(Z); read_item(Y); write_item(Y); read_item(X); write_item(X);	read_item(Y); read_item(Z); write_item(Y); write_item(Z);



Equivalent serial schedule

$T_3 \rightarrow T_1 \rightarrow T_2$

$T_3 \rightarrow T_2 \rightarrow T_1$

CHARACTERIZING SCHEDULES BASED ON SERIALIZABILITY

- Serializability is hard to check.
 - Interleaving of operations occurs in an operating system through some scheduler
 - Difficult to determine beforehand how the operations in a schedule will be interleaved.



CHARACTERIZING SCHEDULES BASED ON SERIALIZABILITY

- Come up with methods (protocols) to ensure serializability.
- It's not possible to determine when a schedule begins and when it ends.
 - Reduce the problem of checking the whole schedule to checking only a **committed part** of the schedule
 - (i.e. operations from only the committed transactions.)
- Current approach used in most DBMSs:
 - Use of locks with two phase locking



TRANSACTION SUPPORT IN SQL2

- A **single** SQL statement is always considered to be **atomic**.
- With SQL, there is no explicit Begin Transaction statement.
 - Transaction initiation is done implicitly when particular SQL statements are encountered.
- Every transaction must have an explicit end statement, which is either a COMMIT or ROLLBACK.



TRANSACTION SUPPORT IN SQL2

Characteristics specified by a SET TRANSACTION statement in SQL2:

▣ Access mode:

- READ ONLY or READ WRITE.
 - ▣ The default is READ WRITE unless the isolation level of READ UNCOMMITTED is specified, in which case READ ONLY is assumed.

▣ Diagnostic size

- specifies an integer value n , indicating the number of conditions that can be held simultaneously
- These conditions supply feedback information (errors or exceptions) to the user on the n most recently executed SQL statement.



TRANSACTION SUPPORT IN SQL2

Characteristics specified by a SET TRANSACTION statement in SQL2 :

- ▣ **Isolation level** <isolation>, where <isolation> can be READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ or SERIALIZABLE. The default is SERIALIZABLE.
 - With SERIALIZABLE: the interleaved execution of transactions will adhere to our notion of serializability.
 - However, if any transaction executes at a lower level, then serializability may be violated.



TRANSACTION SUPPORT IN SQL2

Potential problem with lower isolation levels:

▣ **Dirty Read:**

- Reading a value that was written by a transaction which failed.

▣ **Nonrepeatable Read:**

- Allowing another transaction to write a new value between multiple reads of one transaction.
- A transaction T1 may read a given value from a table. If another transaction T2 later updates that value and T1 reads that value again, T1 will see a different value.
 - ▣ Consider that T1 reads the employee salary for Smith. Next, T2 updates the salary for Smith. If T1 reads Smith's salary again, then it will see a different value for Smith's salary.



TRANSACTION SUPPORT IN SQL2

- Potential problem with lower isolation levels (contd.):

- **Phantoms:**

- New rows being read using the same read with a condition.
 - A transaction T1 may read a set of rows from a table, perhaps based on some condition specified in the SQL WHERE clause.
 - Now suppose that a transaction T2 inserts a new row that also satisfies the WHERE clause condition of T1, into the table used by T1.
 - If T1 is repeated, then T1 will see a row that previously did not exist, called a phantom.



TRANSACTION SUPPORT IN SQL2

□ Sample SQL transaction:

```
EXEC SQL whenever sqlerror go to UNDO;
EXEC SQL SET TRANSACTION
    READ WRITE
    DIAGNOSTICS SIZE 5
    ISOLATION LEVEL SERIALIZABLE;
EXEC SQL INSERT
    INTO EMPLOYEE (FNAME, LNAME, SSN, DNO, SALARY)
    VALUES ('Robert','Smith','991004321',2,35000);
EXEC SQL UPDATE EMPLOYEE
    SET SALARY = SALARY * 1.1
    WHERE DNO = 2;
EXEC SQL COMMIT;
    GOTO THE_END;
UNDO: EXEC SQL ROLLBACK;
THE_END: ...
```



TRANSACTION SUPPORT IN SQL2

Table 21.1 Possible Violations Based on Isolation Levels as Defined in SQL

Isolation Level	Type of Violation		
	Dirty Read	Nonrepeatable Read	Phantom
READ UNCOMMITTED	Yes	Yes	Yes
READ COMMITTED	No	Yes	Yes
REPEATABLE READ	No	No	Yes
SERIALIZABLE	No	No	No

